

Licenciaturas em Engenharia Informática
Programação Web
Ano Lectivo de 2025/2026 – 1º Semestre
AULAS LABORATORIAIS
FICHA 4

Com a aplicação exemplo Gestão de Loja a desenvolver nesta ficha, iniciamos o desenvolvimento da infraestrutura de aplicações base que irão suportar o trabalho prático a realizar nas aulas laboratoriais.

Assim, iremos criar uma App Blazor Web designada por Gestão de Loja, que irá servir de *frontend + backend* para a gestão da loja, isto é, gestão de produtos, de categorias de produtos e dos modos de disponibilização dos produtos. Numa fase posterior, iremos também acrescentar a gestão de encomendas e a gestão de utilizadores e de roles.

Desse modo, execute as seguintes etapas:

1. Criação do Projecto no Visual Studio 2022: Aplicação Gestão de Loja
 - a. 1ª Parte – Criação de uma **Blazor Web App**: Execute o Visual Studio Community 2022, actualizado, e crie um projecto com o template Blazor Web App e com o nome GestaoLoja e com as configurações mostradas nas Figura 1 a 3:

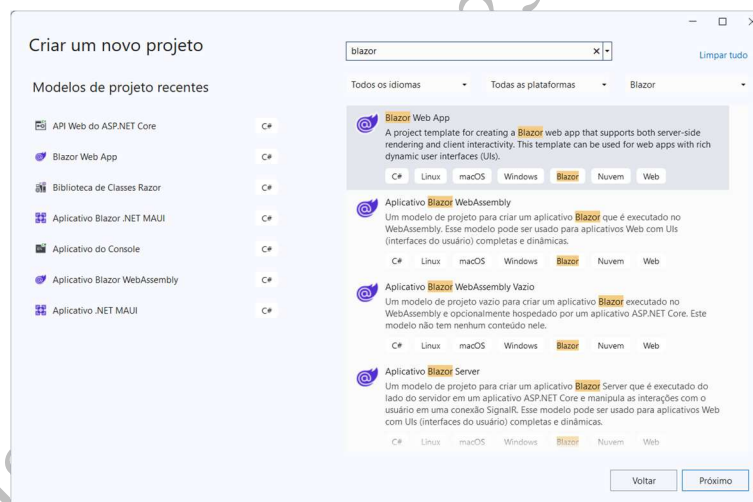


Figura 1- Criar no Visual Studio Community um projecto com o template Blazor Web App

Figura 2- Configurações do projecto Blazor Web App – Passo Intermediário

Figura 3- Configurações do projecto Blazor Web App – Passo Final

- b. Após a criação do projecto Blazor Web App, deverá ver o mesmo que é mostrado na Figura 4.

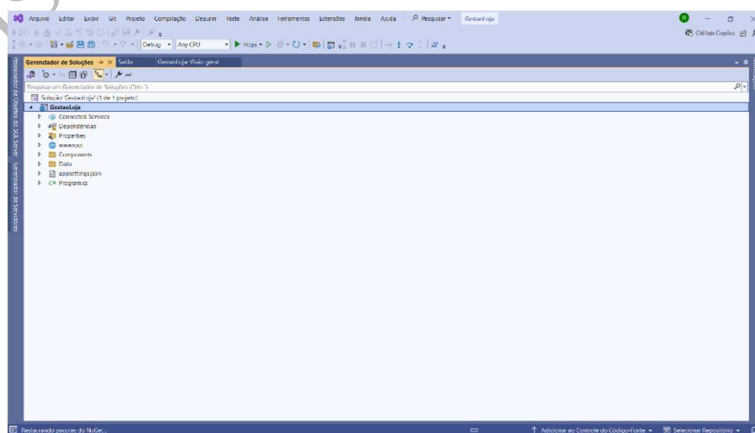


Figura 4 - Projecto Blazor Web App

Compile a aplicação e execute-a devendo ver o mesmo que é mostrado na Figura 5. Verifique que no projecto que acabou de criar, no menú lateral aparecem também as opções para registar utilizadores, *Register*, e para o seu *Login*. Isto deve-se ao facto de termos criado o projecto com a opção “Contas individuais”.

Observe também que foi criada uma directoria *Data* a qual contém, uma subdirectoria chamada *Migrations*, com dois ficheiros de script de construção da Base de Dados no que respeita às tabelas de gestão de *users*. Nesta directoria *Data* foram também criados dois ficheiros, o ficheiro *ApplicationDbContext.cs* e o ficheiro *ApplicationUser.cs*. O professor irá explicar o porquê desses ficheiros e o seu conteúdo.

Observe que, em particular a classe de contexto de base de dados, a classe *ApplicationDbContext* em lugar de considerar como classe base a classe *DbContext*, utiliza uma extensão, derivação, dessa classe a classe base *IdentityDbContext* e uma referência à classe de extensão das propriedades dos utilizadores, a classe *ApplicationUser*. Estas extensões destinam-se a criar toda a infraestrutura *Tabelas da BD + Métodos Específicos*, necessários a que seja utilizada na aplicação criada a Identidade, com recurso ao *Identity Framework* associado ao *Entity Framework*. Também verificamos a criação do ficheiro *appsettings.json* cujo conteúdo também será explicado.

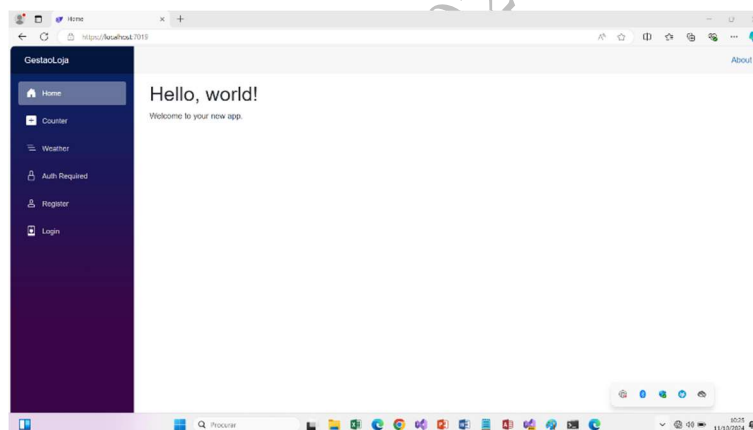


Figura 5- Projecto Blazor Web App – Interface “básica” inicial

2. Antes do passo seguinte, de Customização da aplicação Gestão de Loja, vejamos algumas Convenções para o Relacionamento de Entidades a usar nesta aplicação e que nos permitirá nas classes de domínio *C#* representativas das entidades definidas no MER, também especificar os respectivos relacionamentos.

>> Entity Framework Core – Convenções e Relacionamentos entre entidades

[HTTPS://LEARN.MICROSOFT.COM/EN-US/EF/CORE/MODELING/RELATIONSHIPS](https://learn.microsoft.com/en-us/ef/core/modeling/relationships)

- Por omissão todos os objetos na base de dados são criados no *schema dbo*.
- A EF Core vai criar tabelas na base de dados para todas entidades que tenham definidos na classe de contexto como propriedade DBSet: “*DbSet<TEntity>*”.
- A EF Core também cria tabelas na base de dados para entidades que não estejam definidos como DBSet na classe de contexto, mas que sejam referenciadas por outras entidades DBSet.

- Relacionamentos – definições:

Dependent entity	Entidade que contém a chave estrangeira.
Principal entity	Entidade que contém a chave primária / chave alternativa.
Principal key	Propriedades que identificam exclusivamente a entidade principal. Pode ser a chave primária ou uma chave alternativa.
Foreign key	Propriedades na entidade dependente que são usadas para armazenar os valores de chave principal da entidade relacionada
Navigation property	Propriedade definida na entidade principal e/ou dependente que faz referência à entidade relacionada
	• Collection navigation property: propriedade de navegação que contém referências a muitas entidades relacionadas. • Reference navigation property: propriedade de navegação que contém uma referência a uma única entidade relacionada. • Inverse navigation property: propriedade de navegação na outra extremidade do relacionamento.
Self-referencing relationship	Um relacionamento no qual os tipos de entidade dependente e principal são os mesmos.

- O padrão mais comum para relacionamentos é ter propriedades de navegação definidas em ambas as extremidades do relacionamento e uma propriedade de chave estrangeira definida na classe da entidade dependente.
- No entanto não somos obrigados a definir totalmente as relações. Existem outros padrões/convenções.
- Por exemplo, para definir relacionamentos do tipo 1-N (Um para muitos) podemos usar uma das seguintes convenções:

1. **Convenção 1** - Incluir na entidade dependente uma propriedade de navegação (referência) para a entidade principal.

```
public class Curso{
    public int Id { get; set; }
    public string Nome { get; set; }

    public Categoria categoria { get; set; }
}

public class Categoria{
    public int Id { get; set; }
    public string Nome { get; set; }
}
```

2. **Convenção 2** - Incluir na entidade principal uma propriedade de navegação (coleção).

```
public class Curso{
    public int Id { get; set; }
    public string Nome { get; set; }
}

public class Categoria{
    public int Id { get; set; }
    public string Nome { get; set; }

    public ICollection<Curso> Cursos { get; set; }
}
```

3. **Convenção 3** - Incluir na entidade dependente uma propriedade de navegação (referência) para a entidade principal e incluir na entidade principal uma propriedade de navegação (coleção).

```

public class Curso{
    public int Id { get; set; }
    public string Nome { get; set; }

    public Categoria categoria { get; set; }
}

public class Categoria{
    public int Id { get; set; }
    public string Nome { get; set; }

    public ICollection<Curso> Cursos { get; set; }
}

```

4. **Convenção 4** - Definir a relação na sua totalidade em ambas as entidades, indicando, além as propriedades de navegação, a chave estrangeira na entidade dependente:

```

public class Curso
{
    public int Id { get; set; }
    public string Nome { get; set; }

    public int CategoriaId { get; set; }
    public Categoria categoria { get; set; }
}

public class Categoria
{
    public int Id { get; set; }
    public string Nome { get; set; }

    public ICollection<Curso> Cursos { get; set; }
}

```

Nota: Existem outros tipos de relacionamentos entre entidades que irão ser abordados mais tarde.

3. Customização da aplicação Gestão de Loja

- Na sub-directoria *Pages*, apague os ficheiros *Counter.cs*, *Wheather.cs* e *Error.cs*
- De acordo com as indicações do professor adicione ao projecto a directoria *Entities* e nessa directoria crie as classes de domínio *Categorias.cs*, *Produtos.cs* e *ModoEntrega.cs*
- Altere o conteúdo do ficheiro *ApplicationDbContext.cs* para se considerar na criação da Base de Dados as três classes de domínio atrás criadas.
- De acordo com as orientações dadas, altere a classe *ApplicationUser.cs*
- De acordo com as indicações do professor, adicione à classe *appsettings.json* a *DefaultConnection* onde indicará a Base de Dados a utilizar na aplicação designada por *FrutariaPWeb* e o respectivo caminho e instância do SGBD *SqlServer localDB*.
- Compile a aplicação e caso sejam detectados erros, corrija-os.
- Crie, como indicado pelo professor, a classe *Inicialização.cs* na directoria *Data*

- h. Na raiz do projecto, crie como indicado pelo professor, a classe *AppConfig.cs*
- i. De acordo com as indicações do professor, altere o conteúdo da classe *program.cs*
- j. Compile a solução e se houver erros corrija-os e volte a compilar.
- k. De acordo com as indicações do professor, altere a classe *Program.cs*
- l. Actualize o ficheiro *_Imports.razor* de acordo com as indicações do professor
- m. Compile a solução e verifique se tem erros, corrigindo-os se necessário.
- n. Na directoria *wwwroot*, crie uma subdirectoria designada por *img* e copie para esta subdirectoria os ficheiros de imagem fornecidos via moodle.
- o. Copie para o disco do seu computador as imagens constantes do *moodle* no ficheiro imagens *exemplosimagens.zip*, as quais irão servir mais tarde para criar registos na base de dados.